

Low Level Design (LLD)

Playwright BDD Cucumber Framework · Version 1.1

1. Directory map

src/config	env, artifacts modes, browsers, BrowserStack, run IDs, paths
src/world	CustomWorld (page, logger, shared buffer, stepIndex)
src/hooks	Before/After/AfterStep lifecycle, artifacts, triage
src/pages	BasePage, LoginPage, AddCustomerPage, CustomersPage, OpenAccountPage, CustomerDashboardPage, templates/
src/utils	PlaywrightActions, waits, files, dates
src/errors	FrameworkError hierarchy + mapper
src/logger	Winston factory
src/shared	SharedBuffer (scenario/worker/global)
src/db	MySQL/Postgres adapter, schema, repository
src/reports	Custom HTML, JSON history, extent/ activity HTML
src/ai	llmClient, modelRouter, FailureTriage, runTriage, llmHealth
features/	Gherkin + step definitions (login, customer)
dashboard/	Express API + Chart.js UI
docker/	Dockerfile and compose
.github/	GitHub Actions
jenkins/	Jenkinsfile
docs/	HLD/LLD HTML and PDF

2. Configuration contract

Loaded from .env via src/config/env.ts.

Area	Keys
App	BASE_URL, APP_USERNAME, APP_PASSWORD, APP_ENCRYPTION_KEY
Browser	BROWSER, BROWSER_CHANNEL, HEADLESS, KEEP_BROWSER_OPEN, SLOW_MO, PARALLEL
Artifacts	SCREENSHOT, STEP_SCREENSHOT, SCREENSHOT_FULL_PAGE, VIDEO, TRACE — each off on retain-on-failure
LLM	LLM_ENABLED, LLM_HOST, LLM_API, LLM_API_KEY, LLM_MODEL, LLM_ROUTE, LLM_ROUTER_MODEL, LLM_COMPLEX_MODEL, LLM_PREFER_BILLION, LLM_TIMEOUT_MS. OLLAMA_* remain aliases.
Other	EXECUTION_ENV, DB_*, BROWSERSTACK_*, LOG_LEVEL

Run identity: `RUN_ID` env or file-locked `reports/.run-id` so all workers share one run.

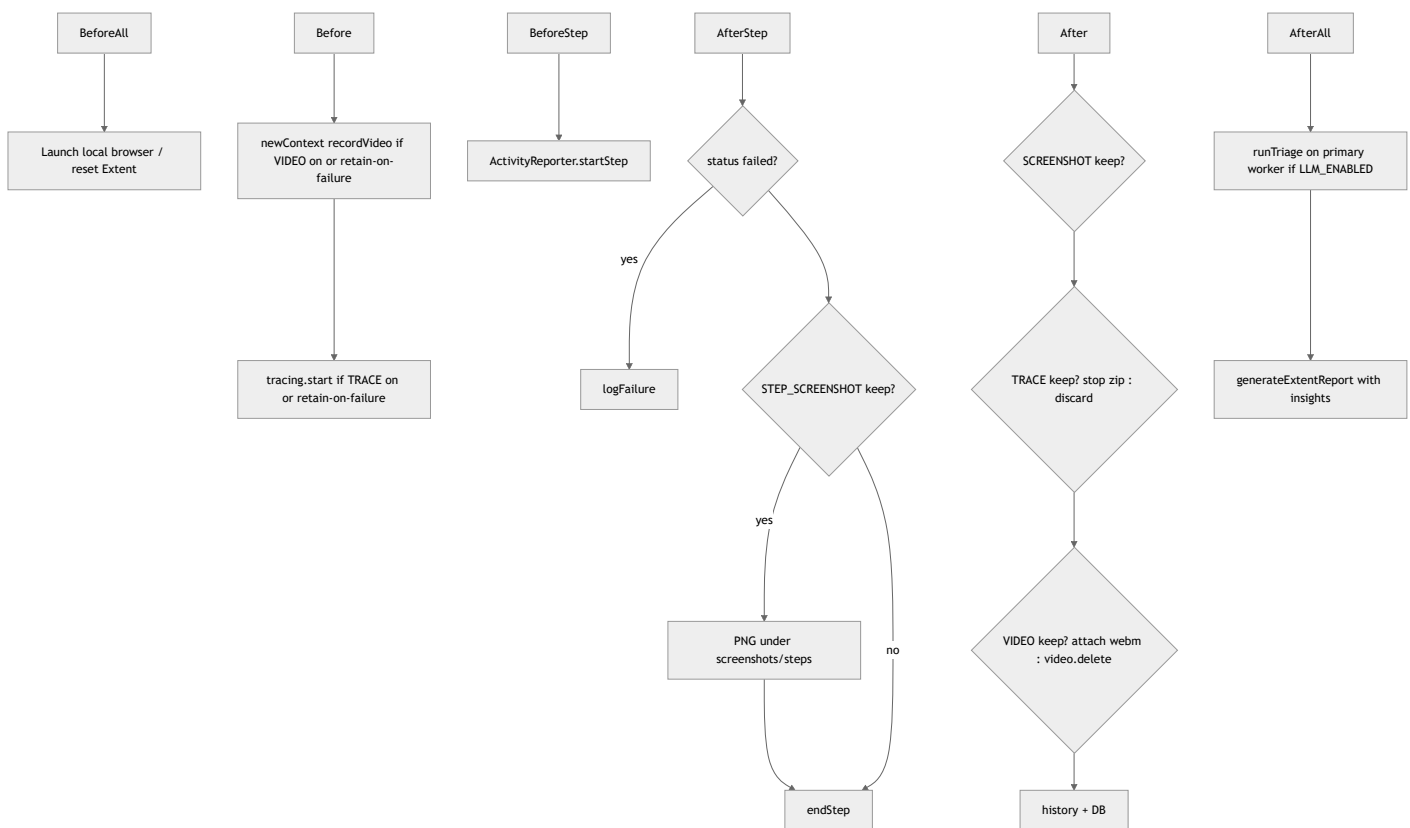
Defaults: `LLM_HOST=http://localaiserver.testingprofessor.net`, `LLM_API=ollama`, `LLM_MODEL=auto`,
`LLM_ROUTER_MODEL=llama3`, `LLM_COMPLEX_MODEL=gemma4:latest`, `LLM_PREFER_BILLION=32`.

3. CustomWorld

browser, context, page, actions, logger, shared
runId, workerId, scenarioId, scenarioName, featureName, tags
stepIndex, screenshotPath, videoPath, tracePath, lastError, browserName

Constructed by Cucumber. Logger is rebound in `Before` with correlation metadata. `stepIndex` increments in `BeforeStep` for step-screenshot file names.

4. Hooks



Hook	Behavior
BeforeAll	Create run, copy Allure categories, reset Extent data, launch local browser (not BrowserStack)
Before	New context/page, optional video dir and tracing, init SharedBuffer scenario scope, start Extent scenario

BeforeStep	Increment <code>stepIndex</code> , start Extent step, log STEP
AfterStep	Log failure on the step; capture step screenshot when <code>STEP_SCREENSHOT</code> applies; end Extent step
After	Scenario screenshot, trace zip or discard, video keep or delete; map error; persist JSON+DB; flush Extent
AfterAll	Close worker browser; finalize run; primary worker runs LLM triage (or custom report); generate Extent HTML; close DB

Artifact helpers: `src/config/artifacts.ts` — `shouldRecordArtifact` / `shouldKeepArtifact`.

5. PlaywrightActions API

All methods log at debug/info, emit Extent activities, and wrap failures with `mapPlaywrightError`.

Methods: `goto`, `click`, `dblClick`, `fill`, `type`, `clear`, `select`, `selectRandom`, `hover`, `check`, `uncheck`, `upload`, `download`, `frame`, `press`, `handleDialog`, `waitForVisible`, `waitForHidden`, `waitForUrl`, `waitForLoadState`, `getText`, `getAttribute`, `getCount`, `isVisible`, `isEnabled`, `screenshot`, `getTitle`, `assertVisible`, `assertText`, `assertTitle`, `assertTitleNotEmpty`, `scrollIntoView`.

`select` accepts a value, `{ label }`, `{ index }`, or blank (random option). `selectRandom` can exclude labels such as `No Category`.

6. Page Object pattern

Core banking pages (not only templates):

Class	Role
LoginPage	Staff sign-in
AddCustomerPage	Create customer from Faker data
CustomersPage	Search / assert customer in list
OpenAccountPage	Open N accounts for a customer
CustomerDashboardPage	Customer login, Continue as first name, deposit

```
export class LoginPage extends BasePage {
  private readonly user = this.page.locator('#user');
  async login(user: string, pass: string) {
```

```

    await this.actions.fill(this.user, 'username', user);
  }
}

```

Feature: `features/customer.feature`. Steps: `features/steps/customer.steps.ts`. Export pages from `src/pages/index.ts`.

7. Error catalog

Class	Code	Category	Typical Playwright signal
<code>ElementNotFoundError</code>	<code>PW_ELEMENT_NOT_FOUND</code>	ELEMENT	Timeout waiting for locator
<code>StrictModeViolationError</code>	<code>PW_STRICT_MODE</code>	ELEMENT	strict mode violation
<code>TimeoutExceededError</code>	<code>PW_TIMEOUT</code>	TIMEOUT	<code>TimeoutError</code>
<code>NavigationError</code>	<code>PW_NAVIGATION</code>	NAVIGATION	<code>net::</code> , <code>NS_ERROR</code> , navigation
<code>TargetClosedError</code>	<code>PW_TARGET_CLOSED</code>	TARGET_CLOSED	Target closed
<code>AssertionFailedError</code>	<code>PW_ASSERTION</code>	ASSERTION	<code>assert*</code> helpers
<code>BrowserLaunchError</code>	<code>PW_BROWSER_LAUNCH</code>	BROWSER	launch / connect failure
<code>ConfigurationError</code>	<code>FW_CONFIGURATION</code>	CONFIGURATION	missing env / bad browser
<code>DatabaseError</code>	<code>FW_DATABASE</code>	DATABASE	SQL / connection
<code>SharedStoreError</code>	<code>FW_SHARED_STORE</code>	SHARED_STORE	file lock
<code>AIServiceError</code>	<code>FW_AI_SERVICE</code>	AI	LLM HTTP / missing API key / timeout
<code>FrameworkError</code>	<code>PW_UNKNOWN</code>	UNKNOWN	fallback

Mapper: `src/errors/mapPlaywrightError.ts`. JSON shape includes code, category, locator, url, action, timestamp, stack, cause.

8. SharedBuffer

```

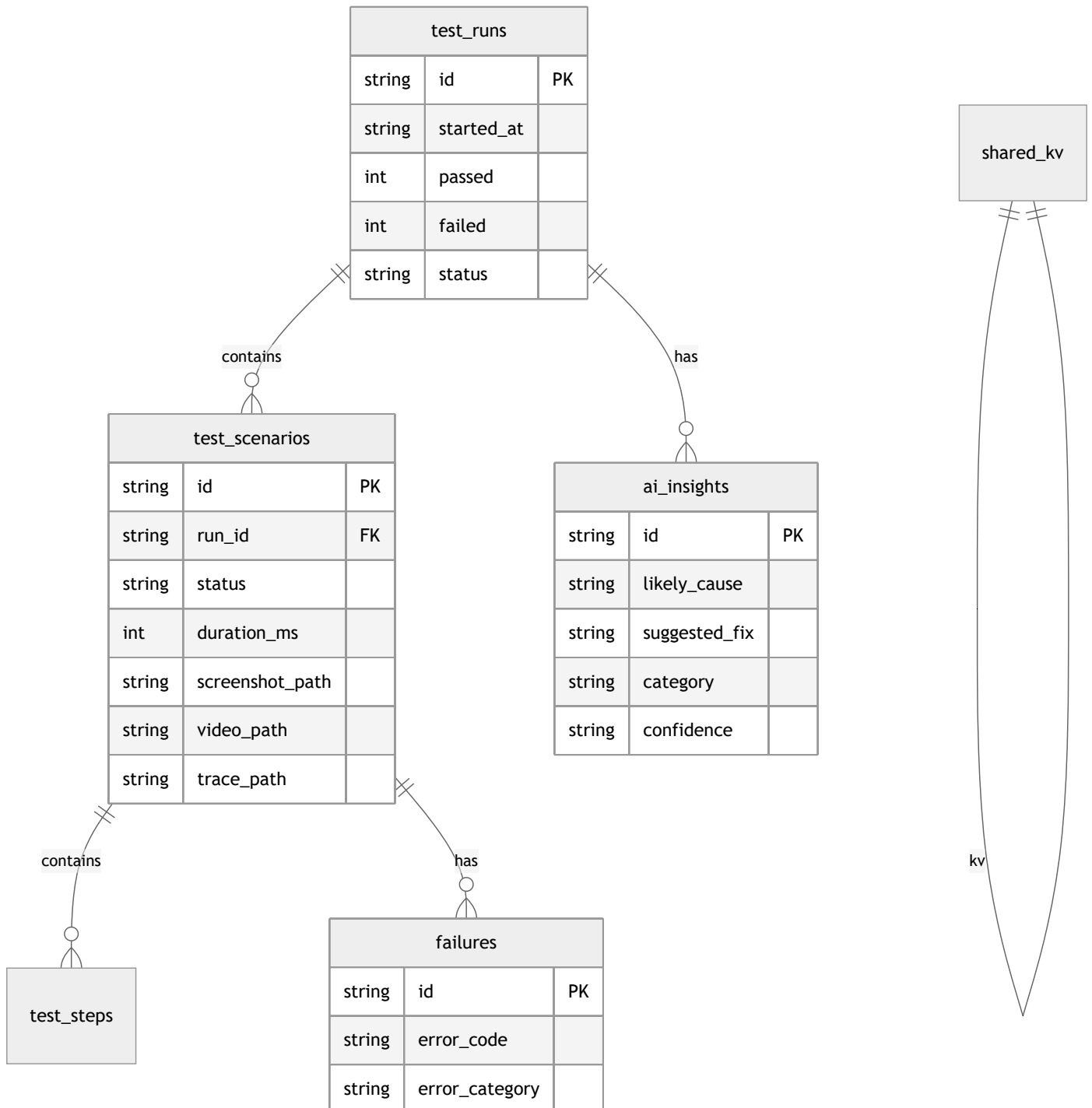
await world.shared.set('orderId', '123', { scope: 'global', ttlMs: 600000 });
const id = await world.shared.get('orderId', 'global');

```

Global writes: `reports/.shared-buffer.json` with `proper-lockfile`, then optional DB upsert into `shared_kv`.

Customer keys: `customer`, `customerName`, `customerFirstName`, `customerLastName`, `customers`, `lastCustomer`, `accounts`, `accountCount`.

9. Database ER

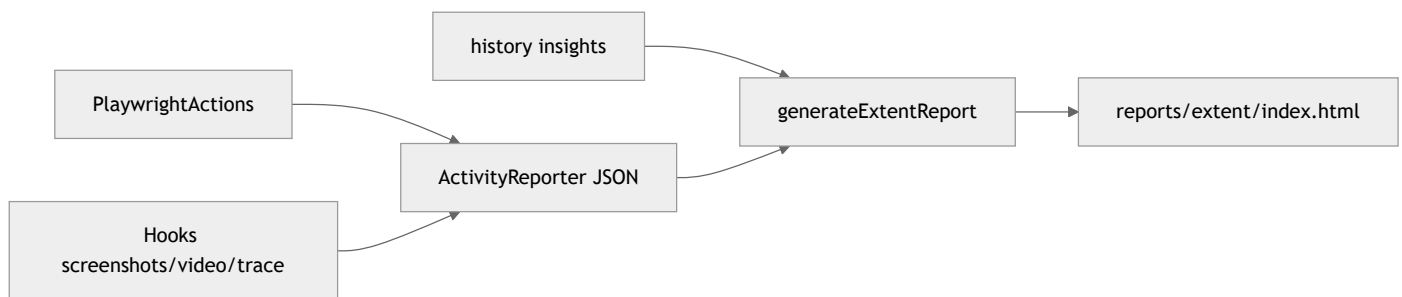


Placeholders use `?`; the Postgres client rewrites them to `$1..$n`. Upsert for `shared_kv` uses `REPLACE INTO` (MySQL) or `ON CONFLICT` (Postgres).

If `DB_ENABLED=false`, repository methods no-op and the dashboard reads `reports/history/*.json`. History insights may include `model` (chosen LLM) even if the DB column is not present.

10. Reporting modules

- `historyStore.ts` — append scenario, insights, maintain runs index.
- `customFailureReport.ts` — HTML with mapped reason, locator, screenshot, AI note + model name.
- `src/reports/extent/` — ActivityReporter (Gherkin steps + Playwright actions + step screenshots), `generateExtentReport.ts` writes `reports/extent/index.html` with video/trace links and AI box.
- Allure formatter in `cucumber.js`; categories copied from `src/reports/allure-categories.json`.



11. Dashboard APIs

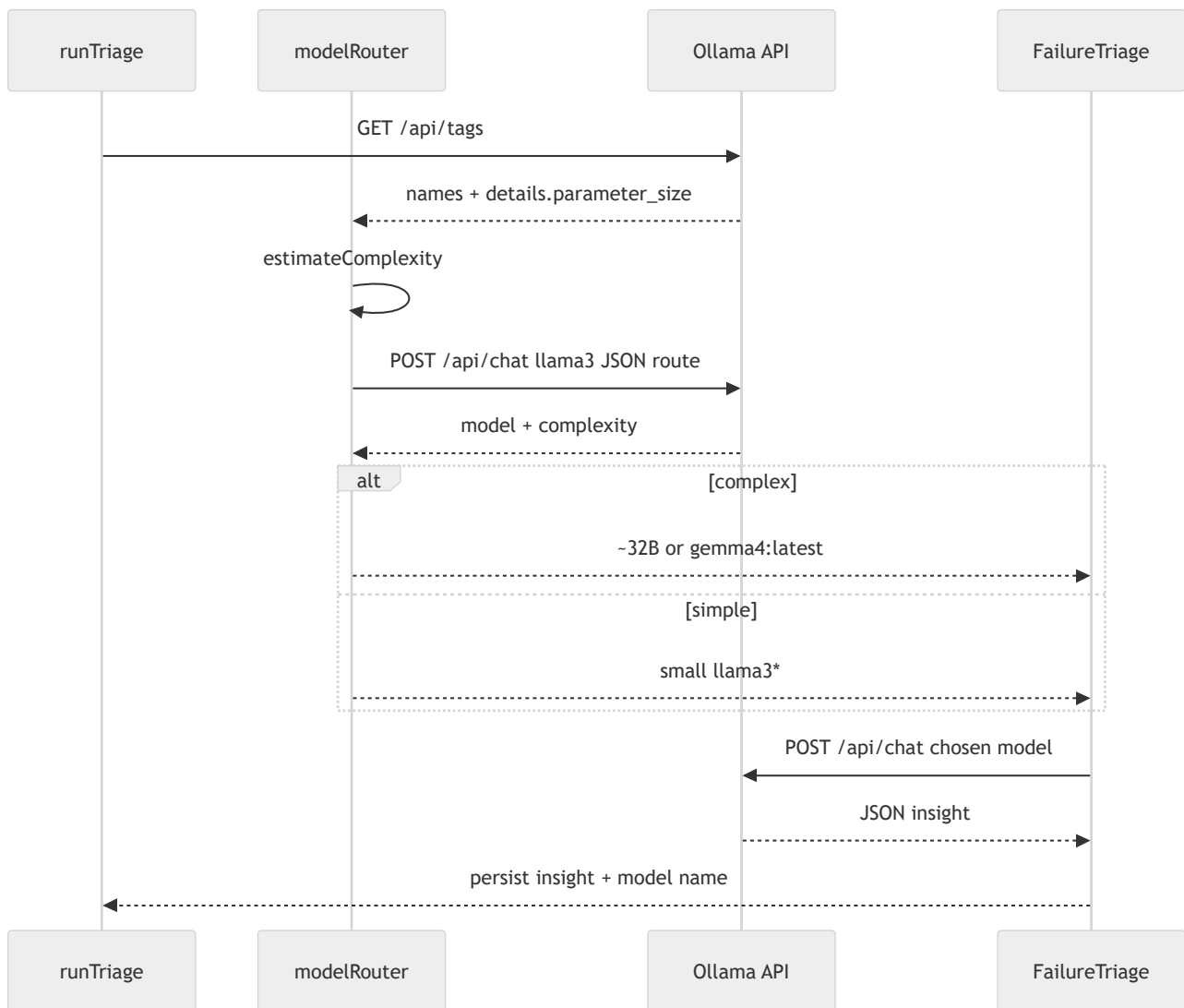
Method	Path	Description
GET	/api/health	DB flag, LLM enabled/reachable, host, model
GET	/api/runs	Run list
GET	/api/runs/:id	Run + scenarios
GET	/api/failures	Optional ?runId=
GET	/api/trends	Last 30 runs for charts
GET	/api/insights	AI insights
POST	/api/triage	Trigger LLM triage for a run

UI: overview KPIs, pass/fail trend (Chart.js), run table, failure drill-down, flaky detection, AI Insights tab showing category, confidence, and model name.

12. LLM triage and routing

Modules:

- `llmClient.ts` — Ollama `/api/tags` and `/api/chat` (OpenAI `/v1/*` fallback), Bearer + `x-API-Key`, timeout, catalog with `parameterBillion`.
- `modelRouter.ts` — estimate complexity; list catalog; ask small llama3 to pick; heuristic if router missing.
- `FailureTriage.ts` — JSON prompt for `likelyCause` / `category` / `suggestedFix` / `confidence`; stores `model`.
- `runTriage.ts` — iterate history failures, skip already-triaged, write custom report.
- `llmHealth.ts` — `npm run ai:health` prints installed models and 32B-class entries.



32B-class band is 27B–40B so names like `qwen2.5:32b`, `llama3.1:32b`, and `gemma2:27b` qualify. Size is taken from `details.parameter_size` or parsed from the tag (`:32b`, `8x7b MoE`). The llama3 router itself is constrained to models $\leq 8B$ so a 32B llama3 is not used for routing.

Heuristic when llama3 is not installed: complex $\rightarrow$ closest $\sim 32B$ then `gemma4:latest`; simple $\rightarrow$ smallest llama3*; medium $\rightarrow \sim 8B$.

Auth: nginx in front of the server returns 401 without `LLM_API_KEY`. Chat is skipped with a warning; reports still generate.

13. BrowserStack connect

```
wss://cdp.browserstack.com/playwright?caps=<urlencoded JSON>
```

Caps include browser, os, build, username, accessKey, scenario name. Local launch path is skipped when `EXECUTION_ENV=browserstack`; each scenario connects and closes its own remote browser.

14. CI/CD

- GitHub Actions: matrix chromium/firefox/webkit, artifact upload, optional BrowserStack job.
- Jenkins: install, optional migrate, test, Allure + HTML publisher.
- Docker: official Playwright image; compose brings MySQL, Postgres, Ollama, dashboard, tests profile. Hosted LLM is selected with `LLM_HOST` / `LLM_API_KEY`.

15. npm scripts (execution styles)

Script	Use
test / test:chrome / firefox / webkit	Browsers
test:headed / test:slow / test:debug / test:keep-open	Local debug
test:parallel	Cucumber workers
test:login / test:customer / test:e2e / test:customer:datadriven	Tag filters
test:browserstack / test:trace	Remote / tracing
report:custom / report:extent / report:all	HTML reports + optional triage
ai:health / ai:triage	List models / run failure triage
dashboard:start / db:migrate	Ops
docker:up / docker:test	Containers
docs:pdf	HLD/LLD PDF via Playwright

Codegen URL: `npm run codegen:url --url=https://corebanking.testingprofessor.net --browser=firefox`

Tags: `npm run test:tags -- @smoke` and not `@wip`

16. Logging format

Console: timestamp, level, `runId|workerId|scenarioId`, message. Files: JSON in `logs/framework.log` and `logs/errors.log` with rotation. Step screenshots and LLM route decisions are logged at info.